JS this Keyword- Practice Questions

1. `this` Inside a Global Scope (Browser Context)

Question:
What does `this` refer to in the global scope inside a browser?

Expected Output (in browser):

```
Window { ... }
```

2. `this` Inside an Object Method

Question:
Demonstrate how `this` works inside an object method.

Input:

```
const person = {
  name: "Alice",
  greet: function() {
    return this.name;
  }
};
```

Expected Output:

```
"Alice"
```

3. `this` Inside a Function (Non-strict Mode)

Question:
What does `this` refer to inside a function in non-strict mode?

Input:

```
function showThis() {
  console.log(this);
}
showThis();
```

Expected Output (in browser):

```
Window { ... }
```

4. `this` Inside a Function (Strict Mode)

Question:
How does `this` behave in a function with strict mode?

Input:

```
"use strict";
function showThisStrict() {
  console.log(this);
}
showThisStrict();
```

Expected Output:

```
undefined
```

5. Arrow Function Inside an Object

Question:
How does `this` behave inside an arrow function inside an object?

Input:

```
const obj = {
  value: 42,
  getValue: () => console.log(this.value)
};
obj.getValue();
```

Expected Output:

```
undefined
```

6. `this` in an Event Listener

Question:
Demonstrate the value of `this` inside an event listener.

Input (in browser console):

```
document.body.addEventListener("click", function() {
  console.log(this);
});
```

Expected Output:

```
<body>...</body>
```

7. `this` Inside a Constructor Function

Question:
What does `this` refer to inside a constructor function?

Input:

```
function Car(name) {
  this.name = name;
}
const myCar = new Car("Tesla");
console.log(myCar.name);
```

Expected Output:

```
"Tesla"
```

8. Arrow Function Inside a Constructor

Question:
How does `this` behave inside an arrow function inside a constructor?

Input:

```
function Car(name) {
  this.name = name;
  this.getName = () => this.name;
}
const myCar = new Car("Ford");
```

Expected Output:

```
"Ford"
```

9. Object Method Calling Another Method

Question:
How does `this` behave when calling another method inside an object?

Input:

```
const user = {
  name: "Bob",
  greet() {
    return `Hello, ${this.name}`;
  },
  farewell() {
    return this.greet() + " - Goodbye!";
  }
};
```

Expected Output:

```
"Hello, Bob - Goodbye!"
```

10. `this` in setTimeout (Browser Context)

Question:
What is the value of `this` inside `setTimeout`?

Input:

```
const obj = {
  message: "Hello",
  showMessage() {
    setTimeout(function() {
      console.log(this.message);
    }, 1000);
  }
};
```

Expected Output:

```
undefined
```

JS this Keyword- Assignment Questions

1. Using `this` in a Nested Function

Question:
How does `this` behave in a nested function inside a method?

Input:

```
const obj = {
  name: "Sophia",
  showName: function() {
    function nested() {
      console.log(this.name);
    }
    nested();
  }
};
obj.showName();
```

Expected Output:

```
undefined
```

2. `this` with Object Destructuring

Question:
How does `this` behave with destructured methods?

Input:

```
const person = {
  name: "Alex",
  getName() {
    return this.name;
  }
};
const { getName } = person;
```

Expected Output:

```
undefined
```

3. `this` in an Object with Arrow Function vs Regular Function

Question:
What is the difference between using a regular function and an arrow function inside an object?

Input:

```
const person = {
  name: "Emma",
  regularFunc: function () {
    console.log(this.name);
  },
  arrowFunc: () => {
    console.log(this.name);
  },
};

person.regularFunc();
person.arrowFunc();
```

Expected Output:

```
"Emma"
undefined
```

4. `this` Inside an Object Constructor Function

Question:
How does `this` behave inside a constructor function?

Input:

```
function Car(brand, model) {
  this.brand = brand;
  this.model = model;
  this.getDetails = function () {
    return `${this.brand} ${this.model}`;
  };
}

const myCar = new Car("Toyota", "Camry");
console.log(myCar.getDetails());
```

Expected Output:

```
Toyota Camry
```

5. Using this in a Class Method Called from Another Function

Question:
What happens when a method is called separately?

Input:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise`);
  }
```

```
}

const dog = new Animal("Buddy");
const speakFunction = dog.speak;
speakFunction();
```

Expected Output:

```
undefined makes a noise
```